Deep Funding Level III: How We Built Our Dependency Weight Model

The Problem in Plain Terms

Every software project depends on other projects. The contest gives us 83 repositories and asks: for each one, how much of its value should flow to each of its dependencies?

We have to put a number (a weight) on every dependency, and for each repository all of its dependency weights have to add up to 1.0. In total that is 3,677 (dependency, repository) pairs to fill in.

A panel of human jurors decided the "true" weights. Our score is how far our numbers are from theirs. Lower is better. The challenge is that we never see the jury's answers for most repositories, so we cannot just memorize them. We have to build something that makes good judgments on repositories we have never scored.

Step 1: We Figured Out Exactly How the Score Is Calculated

Most people optimize blind, because the contest never tells you the exact scoring formula. We decided to pin it down first, because you cannot improve a number you cannot measure.

The contest gives one small public file of real jury answers (L2PublicEval.csv, 162 pairs across 3 repositories). We used it as a test sheet and tried different formulas until our local score matched the score the leaderboard reported for our baseline.

- Our first guess (average error per pair) gave 0.006. Way off from the real leaderboard, so that was wrong.
- The formula that actually matched: for each repository, add up the errors of all its dependencies, then average that total across repositories. That gave 0.3440, which matched our baseline's reported leaderboard score of 0.34 right down to the decimals.

In short:

```
score = average over repositories of ( sum of |our weight - jury weight| )
```

Once we had this, we wrote src/local_score.py, an exact copy of the scorer. Now we could test any idea on our own machine before spending a real submission. This was the single most useful thing we did.

Step 2: We Found Out Where the Error Actually Comes From

With the local scorer, we could see which dependencies hurt our score the most. The answer was clear: in every repository, the top 6 or so dependencies cause 85 to 100 percent of the total error. The hundreds of tiny dependencies are already close to the jury and barely move the score at all.

That changed the whole plan. We did not need to fix thousands of numbers. We needed to get the handful of big, important dependencies right for each repository.

Step 3: We Saw What Was Wrong With the Starting Weights

The contest provides a set of baseline weights derived from funding and usage data (seedReposWithDependenciesAndWeights.json). These are a decent starting point. Their overall shape is right: small dependencies get small weights, big ones get big weights.

But the head was off. The funding data tends to pile too much weight onto one dependency, and it under-rates libraries that are genuinely critical to how a project works simply because they did not receive much historical funding. Funding history is not the same thing as technical importance, and a human juror scores on technical importance.

Step 4: Our Fix, an Expert Juror That Corrects the Head

Instead of throwing away the baseline (which is good in the tail), we keep the tail and fix only the head. This is the core of our method.

For each repository we do this:

1. Take the top dependencies by baseline weight, since those drive the score.
2. Show those dependencies, with their baseline weights, to Claude acting as an expert open-source juror.
3. Ask it to give each one a corrected weight based on one question: how central and irreplaceable is this dependency to what the repository actually does? Things that are core and hard to swap out get more. Things that are easily replaced get less.
4. Keep the baseline weights for the long tail of small dependencies.
5. Normalize everything so the repository's weights add up to 1.0.

One detail matters a lot for fairness. We do not tell the juror which kinds of dependencies to favor. We learned a few patterns from the 3 public repositories, but if we hard-coded those patterns we would just be overfitting to the 3 repositories we can see and likely hurting the 80 we cannot. So the juror judges every dependency on its own merits. That keeps the method general.

How Well It Works

We scored every step locally with the exact metric on the 3 public repositories:

Model	Score (lower is better)
Funding baseline	0.344
Rough manual corrections	0.237
Expert-juror head correction	0.121

Then we confirmed it end to end. We uploaded the corrected submission and the public leaderboard returned 0.1206, matching our local 0.121 almost exactly. For context, the cluster of genuine models near the top sat around 0.18, so our method is comfortably ahead of that group.

Why We Did Not Just Copy the Answer Key

This is the most important thing to understand about our submission.

That public file, L2PublicEval.csv, is also the file the public leaderboard scores against. So anyone can paste those exact numbers into a submission and score close to zero on the public board. Many of the top entries do exactly this. We chose not to, on purpose, because:

- The prize is decided on hidden repositories, where no answer key exists. A copied submission scores near zero on the public 3 and proves nothing about the other 80.
- A pasted answer key shows no method that carries over to data you have not seen. It is a leaderboard trick, not a model.

So our submission carries the juror's real judgment for all 83 repositories: the 3 public ones land around 0.12 (not zero), and the 80 hidden ones use the exact same principled method. We would rather be the strongest honest model than the top fake number.

Being Honest About the Limits

We can only directly check 3 of the 83 repositories, because that is all the public answer key covers. Those 3 prove the mechanism works (0.34 down to 0.12). The other 80 use the same method applied fresh, and we expect it to carry over, but we cannot verify them directly. This is an informed bet built on a validated mechanism, and we want to state that plainly rather than overclaim.

How to Reproduce Our Results

```
pip install -r requirements.txt
python run_ai_juror_pipeline.py
python -c "from src.local_score import score_file; \
           score_file('submission_ai_juror_full.csv')"
```

- data/claude_juror_corrections.py holds the juror's head corrections for every repository.
- run_ai_juror_pipeline.py blends those corrections with the funding baseline and writes the final submission.
- src/local_score.py reproduces the exact leaderboard metric so you can verify the score yourself.

Where the Data Comes From

From the public deepfunding/dependency-graph repository:

- seedReposWithDependenciesAndWeights.json: the funding-derived baseline weights we start from.
- seedReposWithDependencies.json and seedReposWithNoTransitiveDependencies.json:

the dependency structure for each repository.

- L2PublicEval.csv: the public jury weights, which we use only as a held-out test to check our score, never as something to copy into the submission.